Git & GitHub for DevOps — Complete Teaching Package

> ***A beginner-friendly, 5-year-old-understandable, production-ready curriculum and reference for teaching Git & GitHub to DevOps students.***

What Is This?

This is a **complete teaching package** designed for DevOps instructors. It includes:

- **3 Complete Curricula** — Choose the teaching style that fits your class
- **1 DevOps Reference Bible** — A comprehensive manual for students to keep
- **Step-by-step commands** with expected outputs
- **Beginner-friendly analogies** that scale to professional terminology
- **Real DevOps context** — Not just "how Git works" but "how DevOps teams actually use Git"

📁 Directory Structure

```
/home/greg/learnkimchi/
│
├── README.md                          ← You are here (navigation & quick start)
├── git-github-devops-bible.md         ← The Reference Manual
│
├── approach-1-linear/
│   └── curriculum.md                  ← Theory first, then practice
│
├── approach-2-project-driven/
│   └── curriculum.md                  ← One continuous project, start to finish
│
└── approach-3-story-mode/
    └── curriculum.md                  ← DevOps team onboarding story (recommended)
```

🚀 Quick Start — How to Use This Package

For Instructors

Step 1 — Pick your approach based on your students:

If your students are...	Use	Description
Complete beginners who need structure	**Approach 1**	Traditional lecture → lab → exercise format
Hands-on learners who want a real result	**Approach 2**	Students build one portfolio website across all sessions
Easily bored, need engagement/narrative	**Approach 3 (recommended)**	Students role-play joining a DevOps team; each session is a "day at work"

Step 2 — Assign environment setup BEFORE class:
- Students need Git, GitHub account, SSH keys, and a text editor
- This takes 20-30 minutes; don't use class time unless necessary
- See: [Environment Setup] below

Step 3 — Deliver the 5 sessions:
- Each session = exactly 60 minutes
- Each session has: Slides → Lab → Exercise → Review
- All commands include the **expected output** — students know if they're on track

Step 4 — Hand out the Bible:
- `git-github-devops-bible.md` is the reference students keep forever
- It's organized like a textbook: look up concepts, commands, or errors by topic

For Self-Learners / Students

Step 1 — Set up your environment (see below).

Step 2 — Pick one approach and follow it session by session.

Step 3 — Keep `git-github-devops-bible.md` open as your reference.
- When you forget a command, look at the cheat sheet (Part 12)
- When something breaks, check the troubleshooting guide (Part 11)

Step 4 — Build your portfolio.
- By the end, you'll have a GitHub repo with CI/CD — show it in job interviews

---

⚙ Environment Setup (Do This First)

> ***Analogy:*** *Before you can drive, you need a license and keys. This section gets you licensed.*

Step 1: Install Git

If you have Windows

1. Go to `https://git-scm.com/download/win`
2. Download and run the installer
3. Click **Next** repeatedly (defaults are fine)
4. Open **Git Bash** (search Start menu)

Check:

```
$ git --version
```

Expected: `git version 2.43.0...` (or similar)

If you have Mac

1. Open **Terminal** (Cmd+Space, type `terminal`)
2. Run:

```
$ git --version
```

Expected: `git version 2.39...` (or similar)

If it says "not installed," click **Install** when macOS prompts you.

If you have Linux

```
$ sudo apt update && sudo apt install git -y   # Ubuntu/Debian
# OR
$ sudo dnf install git -y                      # Fedora/RHEL

$ git --version
```

Expected: `git version 2.34...` (or similar)

Step 2: Tell Git Who You Are

```
$ git config --global user.name "Your Full Name"
$ git config --global user.email "your.email@example.com"
```

> ***Important:*** *Use the SAME email for GitHub in the next step.*

Verify:

```
$ git config --global user.name
$ git config --global user.email
```

Step 3: Create a GitHub Account

1. Go to `https://github.com`
2. Click **Sign up**
3. Use the **same email** as Step 2
4. Complete verification

Step 4: Set Up SSH Keys (The Secret Handshake)

This lets your computer talk to GitHub without a password.

```
$ ssh-keygen -t ed25519 -C "your.email@example.com"
```

Press **Enter** three times (accept defaults, no password).

Copy your key:
- **Mac:** `pbcopy < ~/.ssh/id_ed25519.pub`
- **Windows (Git Bash):** `cat ~/.ssh/id_ed25519.pub | clip`
- **Linux:** `cat ~/.ssh/id_ed25519.pub` (select and copy)

Add to GitHub:
1. `https://github.com` → **Settings**
2. **SSH and GPG keys** → **New SSH key**
3. Title: `My laptop`
4. Paste the key
5. Click **Add SSH key**

Test it:

```
$ ssh -T git@github.com
```

Type `yes` when asked.

Expected: `Hi yourusername! You've successfully authenticated...`

Step 5: Install a Text Editor

- **Recommended:** VS Code (`https://code.visualstudio.com`)
- **Alternatives:** Notepad++ (Windows), TextEdit (Mac), Nano (Linux)

□ The Three Approaches Explained

Approach 1: Linear Lecture + Lab

Style: Theory first, then practice
Best for: Students who need structure and clear boundaries

Each session:
1. **Lecture** (20 min) — Slides and analogies
2. **Guided Lab** (25 min) — Instructor demos, students follow
3. **Independent Exercise** (10 min) — Students work alone
4. **Review** (5 min) — Q&A

Topics per session:
- Session 1: Git basics (init, add, commit, status, log)
- Session 2: Branching and merging
- Session 3: GitHub, push, pull, Pull Requests
- Session 4: Undoing, reverting, conflict resolution
- Session 5: CI/CD with GitHub Actions

Approach 2: Project-Driven (Build-Along)

Style: One continuous project
Best for: Students who learn by building real things

Students build `my-devops-portfolio` across all 5 sessions:
- Session 1: Create repo, build Home page
- Session 2: Add About + Projects pages using branches
- Session 3: Push to GitHub, add Contact page via PR
- Session 4: Fix a bug, handle a merge conflict
- Session 5: Add CI/CD pipeline that validates the site

By the end, every student has a portfolio repo with CI/CD.

Approach 3: Story-Mode (Recommended)

Style: DevOps team onboarding narrative
Best for: Students who need engagement and context to remember

Students role-play joining "TechCorp" as a new DevOps engineer:
- Session 1: "Day 1 at the Company" — Set up repo, first commits
- Session 2: "Fixing Your First Bug" — Branching to fix a typo
- Session 3: "Submitting Your Work" — Push to GitHub, open first PR
- Session 4: "The Deploy Broke!" — Revert a bad commit, resolve conflicts
- Session 5: "Automating Everything" — Build a CI robot with GitHub Actions

Each session is a chapter in a story. Theory is introduced because the story needs it.

□ The DevOps Bible

`git-github-devops-bible.md` is your **forever reference**. It is organized like a textbook:

Part	Topic	When to Read
Part 0	Environment Setup	Before you start
Part 1	Core Concepts	If you're new to Git
Part 2	Essential Daily Workflow	Every day — the commands you actually use
Part 3	Branching & Merging	When working with branches
Part 4	GitHub Workflow	When pushing code or opening PRs
Part 5	History & Undoing	When you make a mistake
Part 6	Collaboration	When working with a team
Part 7	DevOps Branching Patterns	When designing team workflows
Part 8	CI/CD with GitHub Actions	When automating deploys
Part 9	Release Management	When shipping versions
Part 10	Advanced Topics	When you're comfortable and want more
Part 11	Troubleshooting Guide	WHEN SOMETHING BREAKS
Part 12	Command Cheat Sheet	When you forgot the exact flag

🔄 Common Workflows

The Solo Dev Workflow

```
# Start of day
$ cd my-project
$ git switch main
$ git pull

# Create feature branch
$ git switch -c feature/my-change

# Do work... edit files...

# Commit
$ git add .
$ git commit -m "Clear description of what changed"

# Push
$ git push -u origin feature/my-change

# Open Pull Request on GitHub, merge, delete branch

# Back to main
$ git switch main
$ git pull
$ git branch -d feature/my-change
```

The "Oh No, Production Is Down" Workflow

```
# Find the bad commit
$ git log --oneline

# Revert it
$ git revert BAD_COMMIT_ID --no-edit

# Push the fix
$ git push

# Check that it's fixed
```

The "I Messed Up But Haven't Pushed Yet" Workflow

```
# Wrong files staged
$ git restore --staged wrong-file.txt

# Bad commit message (last commit only)
$ git commit --amend -m "Better message"

# Committed to wrong branch
$ git switch -c correct-branch  # Brings the commit with you
$ git switch wrong-branch
$ git reset --soft HEAD~1
```

The "My Team Has Changed Main" Workflow

```
# Download their changes
$ git switch main
$ git pull

# Update your feature branch
$ git switch feature/my-change
$ git rebase main

# Fix any conflicts, then push
$ git push --force-with-lease
```

FAQ

Q: Do I need to know programming?
A: No. This course assumes zero coding knowledge.

Q: Can I skip the environment setup?
A: No. Everything else depends on it.

Q: Which approach is best?
A: Approach 3 (Story-Mode) is the most engaging for beginners. Approach 1 is best for very structured learners. Approach 2 is best for those who want a portfolio piece at the end.

Q: How long does each session take?
A: Exactly 60 minutes if you follow the pacing.

Q: Can I teach this in one day instead of five?
A: You can try, but beginners need time to absorb. If you must compress, skip the extended exercises and do them as demos.

Q: Do students need a paid GitHub account?
A: No. Everything works on free GitHub accounts.

Q: What if a student uses Windows and another uses Mac?
A: The commands are the same. Only the terminal application differs (Git Bash vs. Terminal).

Q: What if a student gets stuck?
A: Check Part 11 of the Bible (Troubleshooting Guide). Every common error is listed with a fix.

Q: Can I add my own company examples?
A: Absolutely. Replace "TechCorp" in Approach 3 with your company name.

Q: Where do the analogies come from?
A: Each analogy is designed to be understood by a 5-year-old, then mapped to the real DevOps concept.

□ When Something Breaks

Don't panic. Go to `git-github-devops-bible.md` **→ Part 11: Troubleshooting Guide.**

Common issues covered:
- "fatal: not a git repository"
- "Permission denied"
- Merge conflicts
- "Your branch has diverged"
- Accidentally committed secrets
- Failed GitHub Actions workflows

□ Capstone Checklist

By the end of all 5 sessions, every student should be able to demonstrate:

- [ ] Created a Git repository from scratch
- [ ] Made commits with meaningful messages
- [ ] Created and merged branches
- [ ] Connected a local repo to GitHub
- [ ] Pushed and pulled code
- [ ] Opened and merged a Pull Request
- [ ] Reverted a bad commit
- [ ] Resolved a merge conflict
- [ ] Written a GitHub Actions workflow
- [ ] Set up branch protection
- [ ] Explained what CI/CD means using an analogy

□ License & Usage

□ Need Help?

If you're stuck:
1. Check the **Bible** → Part 11 (Troubleshooting)
2. Check the **Bible** → Part 12 (Cheat Sheet)
3. Run `git status` — it usually tells you what to do next

Happy teaching. May all your commits be clean and all your pipelines green.